

Distributed Job Scheduler

Production-Ready Background Task Processing System

GitHub Repository

<https://github.com/Prithwix00/Distributed-Job-Scheduler-Platform-Production-Ready-Background-Task-Processing-System>

Name	Prithwi Raj Mal
Registration No.	RA2311003011662
Department	C-TECH
Branch	CSE CORE

Section 1

Project Overview and Quick Start

Distributed Job Scheduler

A production-inspired platform for reliably running asynchronous background jobs across many workers. It provides authentication and multi-tenant project management, configurable job queues, five job types (immediate, delayed, scheduled, recurring cron, batch), a worker fleet that claims work atomically, a full retry pipeline with a Dead Letter Queue, plus a live operations dashboard.

The design goal was engineering quality over feature count: correct concurrency, a clean relational schema, observable execution and a modular codebase that is easy to reason about.

Table of contents

- [Architecture at a glance](#architecture-at-a-glance)
 - [Quick start (local, no Docker)](#quick-start-local-no-docker)
 - [Quick start (Docker)](#quick-start-docker)
 - [Project layout](#project-layout)
 - [Core concepts](#core-concepts)
 - [API summary](#api-summary)
 - [Testing](#testing)
 - [Documentation](#documentation)
-

Architecture at a glance

Five cooperating services share one PostgreSQL database, which acts as both the system of record and the job queue:

- **API** (FastAPI) serves the REST interface for users and workers.
- **Scheduler** is a single process that owns all time-based transitions:

promoting due jobs, firing cron schedules, reaping dead workers.

- **Workers** are a horizontally scalable fleet. Each polls the API, claims jobs atomically, runs them concurrently, heartbeats and shuts down gracefully.

- **Frontend** (React) is the operations console.
- **PostgreSQL** stores everything and provides the row-level locking that makes

claiming safe.

The full diagram and control flow live in docs/ARCHITECTURE.md.

Quick start (local, no Docker)

Runs against SQLite so no database server is needed. Python 3.11+ and Node.js must be installed.

RUN in ONE Click

Run the file: `run-local.bat`

```
.\run-local.bat
```

That opens the API, scheduler, worker and dashboard automatically. When ready:

- Dashboard: <http://localhost:5173>
- API docs: <http://localhost:8000/docs>

Demo login (for better showcase)

Seed the demo data first (optional — you can also create a fresh account):

```
cd backend && .venv\Scripts\python.exe -m app.runtime.seed
```

Then sign in with:

```
Email:    demo@example.com
Password: password123
```

You can also register a brand new account directly on the dashboard.

PowerShell alternative

```
powershell -ExecutionPolicy Bypass -File .\run-local.ps1
```

Manual (any OS): four terminals

```
# 1. install
cd backend && python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt

# 2. API (terminal 1)
DATABASE_URL=sqlite:///./dev.db uvicorn app.main:app --reload

# 3. Scheduler (terminal 2)
DATABASE_URL=sqlite:///./dev.db python -m app.runtime.scheduler

# 4. Worker (terminal 3)
DATABASE_URL=sqlite:///./dev.db python -m app.runtime.worker --base-url http://localhost:8000
```

```
# 5. Frontend (terminal 4)
cd frontend && npm install && npm run dev
```

Quick start (Docker)

Requires Docker Desktop with Compose v2. Start Docker Desktop first, wait until the engine is running, then:

```
cp .env.example .env
docker compose up --build
```

This starts Postgres, runs database migrations, then brings up the API, the scheduler, two workers and the dashboard. When it is running:

- Dashboard: <http://localhost:5173>
- API docs (OpenAPI/Swagger): <http://localhost:8000/docs>

Seed demo data:

```
make seed
```

Then sign in with `demo@example.com` / `password123`.

Two worker replicas run by default so you can watch distributed, contention-free claiming on the Workers page. Scale them live:

```
docker compose up --scale worker=5
```

Project layout

```

scheduler/
  backend/
    app/
      config.py           settings (env-driven)
      database.py         engine + session, dialect-aware
      security.py         JWT + password hashing
      core.py             structured logging + typed errors
      deps.py             auth, pagination, tenancy scoping
      main.py             FastAPI app, middleware, error handlers
      models/             SQLAlchemy models (the 12 tables)
      schemas/            Pydantic request/response models
      services/           business logic
        retry.py          backoff computation (pure functions)
        claiming.py       atomic claim + lease reaping
        job_service.py    lifecycle transitions, DLQ handoff
        scheduling.py     promotion, cron firing, worker reaping
      routers/            HTTP endpoints per resource
      runtime/
        executor.py       task registry + demo handlers
        worker.py         the worker process
        scheduler.py      the scheduler process
        seed.py           demo data
      migrations/         Alembic migrations
      tests/              pytest suite (31 tests)
    frontend/             React + Vite + TypeScript dashboard
  docs/                  architecture, ER diagram, API, design decisions
  docker-compose.yml
  Makefile

```

Core concepts

Job lifecycle. A job moves through `QUEUED` -> `CLAIMED` -> `RUNNING` -> `COMPLETED`. On failure it becomes `SCHEDULED` for a retry after a backoff delay, or `DEAD` once retries are exhausted (which also writes a Dead Letter Queue entry). Delayed, scheduled and dependent jobs start in `SCHEDULED` and the scheduler promotes them to `QUEUED` when due.

Atomic claiming. The core correctness property is that each job runs on exactly one worker. Claiming locks candidate rows with `FOR UPDATE SKIP LOCKED` so concurrent workers take disjoint sets and never collide. See `docs/DESIGN_DECISIONS.md`.

Leases and recovery. A claimed job carries a lease that its worker renews on every heartbeat. If a worker dies, the lease expires and the scheduler requeues the job so another worker can pick it up. Idempotent handlers make that re-run safe.

Retry strategies. Fixed, linear and exponential backoff, each with optional jitter to avoid retry storms. The policy is snapshotted onto the job at enqueue time so later policy edits never rewrite in-flight jobs.

API summary

All endpoints are under `/api/v1`. Full reference in docs/API.md.

Area	Endpoints
Auth	POST /auth/register, POST /auth/login, GET /auth/me
Projects	GET/POST /projects, GET /projects/{id}
Queues	POST /projects/{id}/queues, GET /queues/{id}, PATCH /queues/{id}, pause/resume, GET /queues/{id}/stats
Jobs	POST /queues/{id}/jobs, POST /queues/{id}/batches, GET /queues/{id}/jobs (filter + paginate), GET /jobs/{id}, cancel, retry
Schedules	POST/GET /queues/{id}/schedules, toggle
Dead letters	GET /queues/{id}/dead-letters, POST /dead-letters/{id}/replay
Workers	register, heartbeat, claim, start, result, GET /workers
Dashboard	GET /dashboard/overview, GET /dashboard/throughput

Interactive OpenAPI docs are served at `/docs`.

Testing

```
cd backend && python -m pytest
```

The suite covers the parts most likely to break: retry backoff maths, atomic claiming (including a threaded no-double-claim test with eight concurrent workers), lease reaping, the full lifecycle through completion, retry and Dead Letter Queue, cron firing, dependency release plus the REST surface end to end through a `TestClient`. See docs/DESIGN_DECISIONS.md for what is tested and why.

Documentation

- docs/ARCHITECTURE.md: components, control flow, reliability model, scaling.
- docs/ER_DIAGRAM.md: entity relationship diagram, keys, indexes, normalization, cascades.
- docs/API.md: full REST reference with request and response shapes.
- docs/DESIGN_DECISIONS.md: major trade-offs and the reasoning behind them.

Section 2

System Architecture

Architecture

System overview

The platform is composed of five services around a single PostgreSQL database. Postgres is deliberately both the system of record and the queue substrate: its row-level locking (`FOR UPDATE SKIP LOCKED`) is what makes distributed claiming correct without a separate broker.

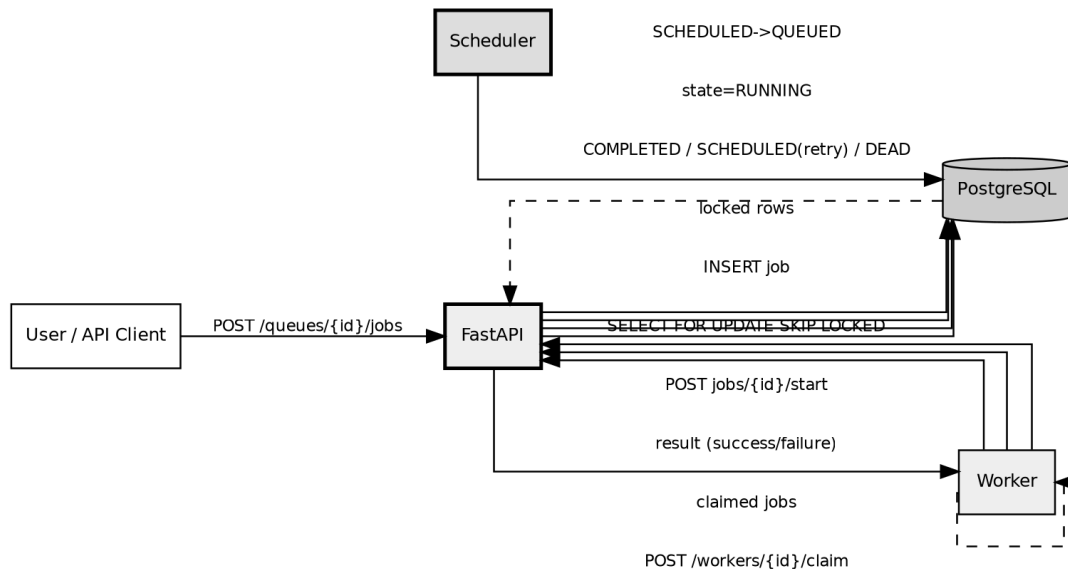


Figure 1: Job control flow

Workers talk to the database through the API rather than connecting directly. This keeps a single authorization and validation boundary, lets workers run anywhere that can reach the API over HTTP and keeps the worker process thin.

Component responsibilities

API (FastAPI). Stateless HTTP layer. Handles auth, tenant scoping, validation, pagination and the worker-facing claim/heartbeat/result endpoints. Every state change routes through the service layer so invariants live in one place. It scales horizontally behind a load balancer because it holds no local state.

Scheduler. A single-instance loop that owns time. On each tick it promotes `SCHEDULED` jobs whose `run_at` has arrived into `QUEUED`, materialises a job for every cron schedule that is due, marks workers whose heartbeat expired as dead and requeues jobs whose leases lapsed. Running exactly one scheduler avoids two clocks disagreeing. If it stops, nothing is lost or duplicated; promotion simply pauses until it restarts.

Worker fleet. Horizontally scalable. Each worker registers, then loops: claim a batch of jobs (bounded by free capacity), dispatch each to a thread pool, report results. A background thread heartbeats every few seconds, which also renews the lease on every job the worker still owns. On `SIGTERM/SIGINT` it stops claiming and drains in-flight work before exiting.

Frontend. React operations console. Polls the API for overview metrics, queue stats, the job explorer, worker liveness and a throughput chart. Polling was chosen over WebSockets for operational simplicity; the trade-off is discussed in the design decisions.

PostgreSQL. Single source of truth. Holds identity, configuration, jobs, execution history, logs, worker state and the Dead Letter Queue.

Control flow: a job from submit to done

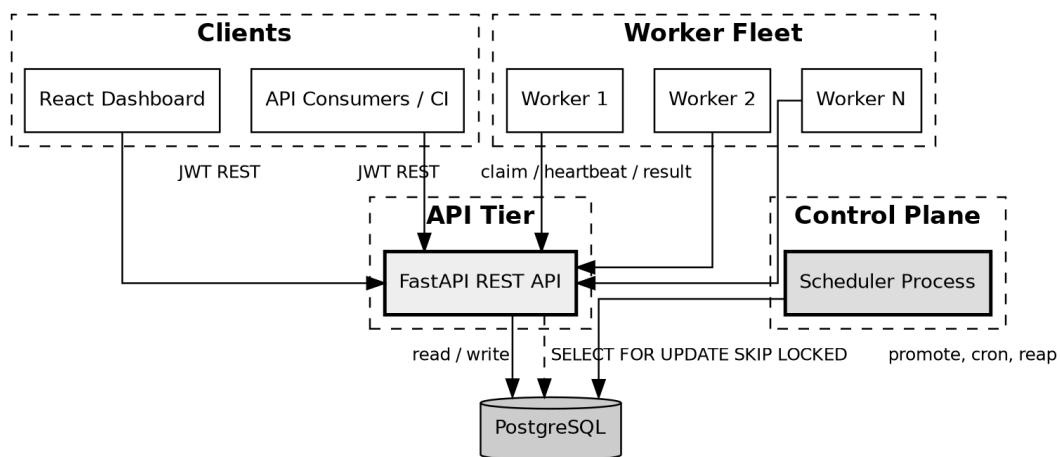


Figure 2: System component overview

Reliability model

The system is designed around one hard invariant and one recovery mechanism.

Exactly-once claiming (the invariant). A `QUEUED` job is handed to exactly one worker. Concurrent claimers lock disjoint candidate rows with `SKIP LOCKED`, so no two workers ever see the same row as claimable. The state transition itself is a guarded `UPDATE ... WHERE state = 'queued'`, so even without row locks (SQLite) only one writer can win.

Lease-based recovery (the safety net). Exactly-once *delivery* is impossible in the presence of crashes, so the system targets at-least-once execution with idempotent handlers. Each claimed job carries a lease that its owner renews on every heartbeat. A worker that crashes stops renewing; the scheduler's `reap_expired_leases` returns the job to `QUEUED` for another worker. Because handlers are written to be idempotent, a re-run after a crash is safe.

Concurrency limits. Each queue has a `concurrency_limit`. The claim query counts jobs already `CLAIMED` or `RUNNING` for the queue and never claims beyond the remaining capacity, so a queue cannot exceed its configured parallelism even across many workers.

Idempotency at submission. An optional `idempotency_key` is unique per queue, so a retried submission collapses to the existing job instead of creating a duplicate.

Scaling and failure behaviour

Concern	Behaviour
More throughput	Add worker replicas. SKIP LOCKED means added workers take disjoint rows with no coordination.
API load	API is stateless; run multiple instances behind a load balancer.
Scheduler failure	Time-based promotion pauses until restart. No data loss or duplication. Jobs already queued still run.
Worker crash	Lease expires, scheduler requeues the job, another worker runs it.
Database as bottleneck	The claim is a single indexed, short transaction. Vertical scaling and read replicas for dashboards handle large loads; the design notes cover sharding by queue as the next step.
Poison job	Retries with backoff, then Dead Letter Queue for human triage and replay.

Observability

Every attempt is recorded as a `JobExecution` row with duration and status, so retry history and metrics are queryable rather than lost in logs. Structured JSON logs carry a request id through the API. `JobLog` rows capture per-job, per-attempt events. The dashboard reads aggregate counts and a bucketed throughput series for at-a-glance health.

Section 3

Database Design and ER Diagram

Database Design

The schema is normalised to third normal form, scoped by tenant and indexed for the two hot paths: claiming jobs and rendering the dashboard.

Entity relationship diagram

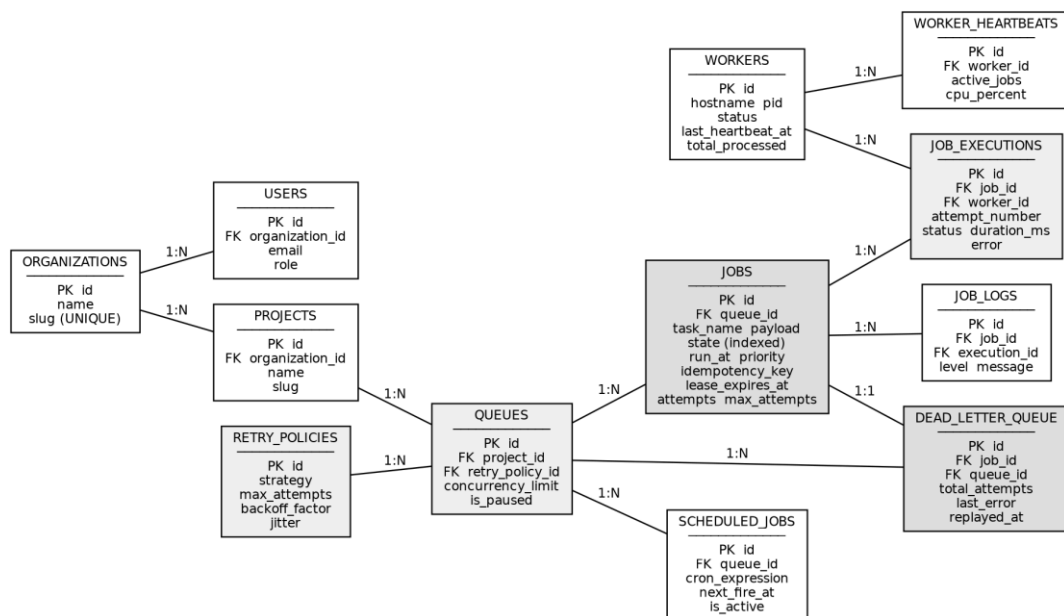


Figure 3: Entity relationship diagram — all 12 tables

Tables and keys

Twelve tables. Every primary key is a UUID string, which keeps ids opaque, non sequential (no enumeration or capacity leakage) and safe to generate on the client or across shards without coordination.

Table	Purpose	Notable foreign keys	On delete
organizations	Top-level tenant		
users	Members of an org	organization_id	CASCADE
projects	A workspace owning queues	organization_id	CASCADE
retry_policies	Reusable backoff config		
queues	A job queue with priority, concurrency, retry policy	project_id, retry_policy_id	CASCADE / SET NULL
jobs	The central work item	queue_id, depends_on_job_id	CASCADE / SET NULL
job_executions	One row per attempt	job_id, worker_id	CASCADE / SET NULL
job_logs	Structured per-attempt log lines	job_id, execution_id	CASCADE
scheduled_jobs	Cron definitions	queue_id	CASCADE
dead_letter_queue	Exhausted jobs for triage	job_id, queue_id	CASCADE
workers	Worker registry		
worker_heartbeats	Liveness and load samples	worker_id	CASCADE

Cascading behaviour

Deletes follow ownership. Removing an organization removes its users, projects, queues and all downstream jobs, executions, logs, schedules and dead letters through `ON DELETE CASCADE`. Two relationships intentionally use `SET NULL` instead: a queue's `retry_policy_id` (deleting a shared policy should not delete the queue) and a job's `depends_on_job_id` and an execution's `worker_id` (history should survive the deletion of a parent job or a retired worker).

Indexing strategy

The claim path is the hottest query in the system, so it gets a dedicated composite index:

- `ix_jobs_claim (queue_id, state, priority, run_at)` supports the exact filter

and ordering the claim uses: find `QUEUED` rows for a queue, highest priority and oldest `run_at` first. This keeps claiming logarithmic even when millions of terminal-state rows sit in the table.

- `ix_jobs_state_runat (state, run_at)` serves the scheduler's promotion scan

across all queues.

- `ix_jobs_batch (batch_id)` groups batch siblings.
- `uq_job_idem (queue_id, idempotency_key)` enforces submission idempotency.
- `ix_exec_job (job_id, attempt_number)` renders a job's ordered attempt history.
- `ix_joblog_job_ts (job_id, created_at)` streams a job's logs in order.
- `ix_sched_next_fire (is_active, next_fire_at)` finds due cron schedules.
- `ix_worker_status_seen (status, last_heartbeat_at)` finds stale workers to reap.
- `ix_dlq_queue (queue_id, created_at)` paginates the Dead Letter Queue.

Uniqueness constraints double as indexes: org slug, project slug per org, queue name per project, user email per org.

Normalization

The schema is in third normal form. Reusable retry configuration lives in its own `retry_policies` table rather than being duplicated on every queue. Each execution attempt is its own row rather than a JSON blob on the job, which keeps attempt history queryable and metrics aggregatable.

One deliberate denormalization exists on `jobs`: the effective retry parameters (`max_attempts`, `retry_strategy`, `base_delay_seconds` and so on) are snapshotted from the queue's policy at enqueue time. This is not redundancy for its own sake; it guarantees that editing a queue's policy never rewrites the backoff behaviour of jobs already in flight. It also removes a join from the hot failure path.

Performance considerations

- ****Short claim transactions.**** The claim locks only the rows it will take and commits immediately, so lock hold time is minimal and `SKIP LOCKED` lets other workers proceed in parallel.
- ****JSONB on Postgres.**** Payloads, results and log context use `JSONB` on Postgres (indexable, operator-rich) and plain `JSON` on SQLite through a dialect variant.
- ****Bounded growth.**** Terminal jobs, executions and heartbeats accumulate. In production these are archival candidates; a periodic job can move old completed rows to cold storage. The composite claim index ensures active claiming stays fast regardless of how much terminal history has built up.
- ****Read/write split.**** Dashboard aggregates are read-only and can be served from a replica so reporting never contends with claiming.

Migrations

Schema changes are managed with Alembic. The initial migration under `backend/migrations/versions/` is generated from the model metadata and creates all twelve tables with their indexes and constraints. The Docker `api` service runs `alembic upgrade head` before serving, so a fresh deployment provisions the schema automatically. Local SQLite development can also fall back to `create_all` on startup for zero-setup iteration.

Section 4

Design Decisions and Trade-offs

Design Decisions

This document records the major trade-offs and the reasoning behind each. The guiding principle throughout was correctness and clarity over feature count.

1. PostgreSQL as the queue, not a dedicated broker

Decision. Use Postgres as both the system of record and the job queue, rather than pairing a database with Redis, RabbitMQ or Kafka.

Why. A database-backed queue gives transactional consistency between the job and its metadata for free: enqueue, state change, execution record and Dead Letter handoff all commit atomically in the same store. There is no dual-write problem where the broker and the database can disagree after a partial failure. For the throughput this system targets, `FOR UPDATE SKIP LOCKED` is a proven pattern (it is how tools such as Que, Solid Queue and River work).

Trade-off. A dedicated broker scales to higher raw message rates and offers push delivery. If claim throughput ever became the bottleneck, the next step is queue sharding (below) or moving the hottest queues onto a streaming broker while keeping the database as the record of truth. The schema and service boundaries are structured so that swap would touch only the claiming service.

2. `FOR UPDATE SKIP LOCKED` for atomic claiming

Decision. Claim jobs by selecting candidate rows with `SELECT ... FOR UPDATE SKIP LOCKED`, then flipping their state inside the same transaction.

Why. This is the crux of correctness. `SKIP LOCKED` means each concurrent worker locks a disjoint set of rows and skips anything another worker already holds, so N workers claim in parallel with no coordination and no job is ever handed to two workers. It avoids both the lost-update race of a naive read-then-write and the throughput collapse of a coarse lock.

Portability. SQLite has no `SKIP LOCKED`. The claim is therefore also expressed as a guarded `UPDATE ... WHERE state = 'queued'`; only the writer whose update changes the row (rowcount one) has claimed it. On Postgres the lock guarantees the win and the guard is belt-and-braces; on SQLite the guard is the sole arbiter and SQLite's serialised writes make it correct. The same code path is exercised by an eight-worker threaded test that asserts no double claim.

3. Workers reach the database through the API

Decision. Workers call HTTP endpoints (`claim`, `start`, `result`, `heartbeat`) instead of opening their own database connections.

Why. One authorization and validation boundary, one place where invariants are enforced. Workers can run anywhere with network reach to the API, in any language, without database credentials. The worker process stays thin and easy to reason about.

Trade-off. An extra network hop per operation versus a direct connection. For background job execution the handler runtime dominates, so the hop is negligible. If it ever mattered, the claim endpoint could be collapsed into a worker library that talks to the database directly using the exact same service function.

4. At-least-once execution with idempotent handlers

Decision. Target at-least-once execution recovered by leases and require handlers to be idempotent, rather than chasing exactly-once.

Why. Exactly-once delivery is not achievable in a distributed system with crashes. A worker can finish a job and die before recording the result and no protocol removes that window. So the honest model is at-least-once: each claimed job carries a lease that the worker renews on every heartbeat, a crashed worker stops renewing and the scheduler requeues the job when the lease expires. Correctness then depends on handlers being safe to re-run, which the demo handlers are (pure functions or keyed on a message id).

Trade-off. Handler authors carry the idempotency burden. The `idempotency_key` on submission and the keyed demo handlers show the intended pattern.

5. Retry policy snapshotted onto the job

Decision. Copy the effective retry parameters from the queue policy onto the job at enqueue time instead of reading the live policy on each failure.

Why. A job that is already retrying should keep the behaviour it started with. If an operator tightens a queue's `max_attempts` while jobs are mid-flight, those jobs should not suddenly dead-letter early. Snapshotting makes retry behaviour deterministic per job and removes a join from the hot failure path.

Trade-off. Policy edits apply only to newly enqueued jobs. That is the desired semantics here; a bulk re-apply could be added if needed.

6. A single scheduler process

Decision. Run exactly one scheduler that owns promotion, cron firing and reaping.

Why. Time-based transitions want a single clock authority. One scheduler means no two processes disagree about what is due. It is a small, stateless loop whose only failure mode is that promotion pauses until it restarts, with no data loss and no duplication.

Trade-off. It is a single point for time-based work. It is cheap to run under a process supervisor that restarts it. For high availability, a leader election (advisory lock in Postgres) would let a standby take over; the tick logic is already idempotent, so that upgrade is low risk.

7. Polling over WebSockets for the dashboard

Decision. The dashboard polls the API on short intervals rather than subscribing over WebSockets.

Why. Polling is stateless on the server, trivial to reason about, resilient to reconnects and entirely adequate for an operations console refreshing every few seconds. It kept the API stateless and the frontend simple.

Trade-off. Slightly higher request volume and a few seconds of latency versus instant push. For a dashboard that is a good trade. WebSocket live updates are noted as a natural extension.

8. UUID primary keys

Decision. String UUIDs for every primary key.

Why. Ids are opaque and non sequential, so they leak neither record counts nor an enumeration surface. They can be generated on the client or across shards without a central sequence, which matters for the sharding path.

Trade-off. Wider keys than integers and non-monotonic inserts. The composite claim index keeps the hot path fast regardless and the operational benefits outweigh the storage cost here.

9. One codebase, SQLite for dev and Postgres for production

Decision. The same application runs on SQLite locally and Postgres in production through a dialect-aware engine and claim.

Why. Zero-setup local development and a fast, dependency-free test suite, without a second implementation to keep in sync. Contributors run `pytest` with no database server; production gets real row-level locking.

Trade-off. The claim carries two code paths. They are small, share the same guarded update and are both covered by tests, so the maintenance cost is low.

10. Thread pool for worker concurrency

Decision. Each worker runs jobs in a `ThreadPoolExecutor` sized to its concurrency.

Why. Background job handlers are typically IO-bound (network calls, database writes), where threads give real concurrency simply. It also lets handlers be ordinary synchronous functions, which is the most common shape for real tasks.

Trade-off. CPU-bound handlers would be limited by the GIL. For those, workers scale out as separate processes (the compose file runs two by default and scales further), which is the horizontal model the whole system is built around anyway.

Testing strategy

Tests target the parts where a bug is most costly, not line coverage for its own sake.

- **Retry maths** are pure functions, tested exhaustively across strategies, clamping and jitter bounds.
- **Atomic claiming** has a threaded test with eight concurrent workers claiming two hundred jobs, asserting every job is claimed exactly once. Separate tests cover concurrency-limit capping, pause, future-dated jobs, priority ordering and lease reaping of dead workers.
- **Lifecycle** is driven end to end: create, claim, start, complete; and the failure path through retry into the Dead Letter Queue, asserting attempt counts, execution history and the DLQ entry.
- **Scheduling** covers promotion of due jobs, cron firing with schedule advancement and dependency release on parent completion.
- **The REST surface** is tested through a `TestClient`, including a full worker pipeline (register, claim, start, result) that exercises the exact endpoints the worker process calls.

Run with `cd backend && python -m pytest`.

Section 5: Screenshots

The screenshots below demonstrate the running system. They cover the main dashboard, queue management, job detail view, worker monitoring and the Dead Letter Queue triage screen.

1. Dashboard Overview

System health, job state counts and throughput chart (<http://localhost:5173>)

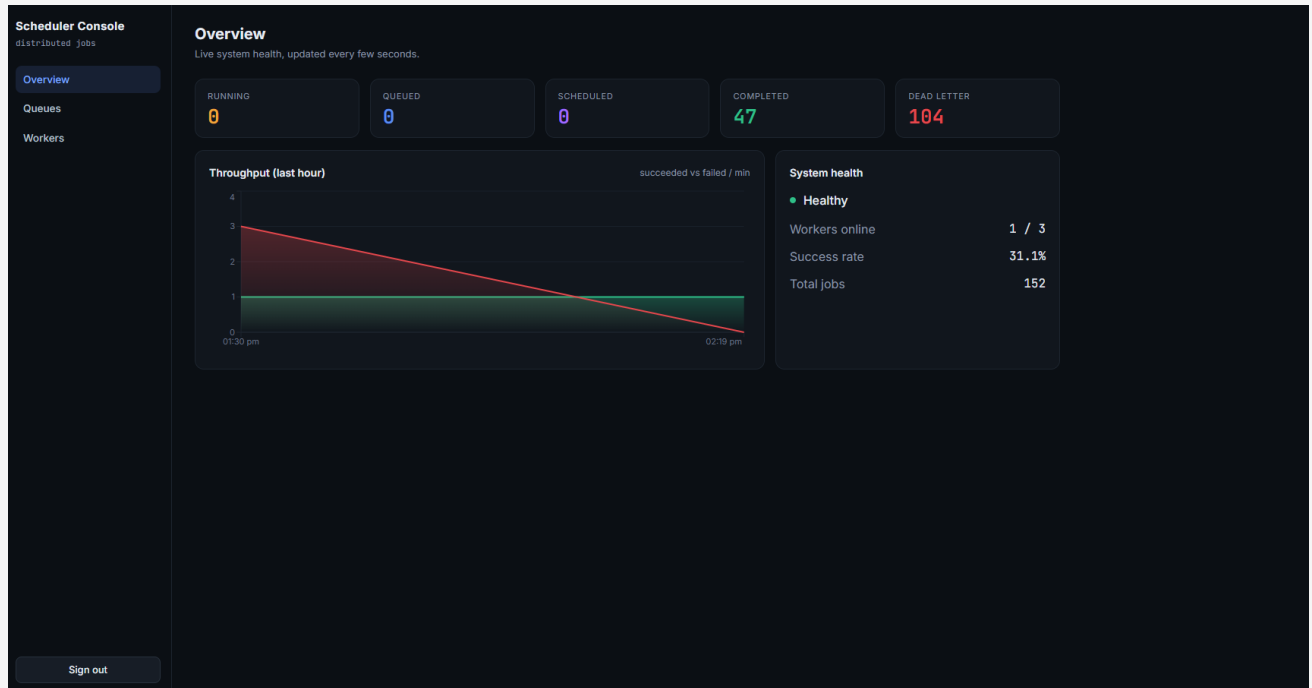


Figure 1: Dashboard Overview

2. Queue Management

Project and queue list with priority, concurrency and retry policy configuration

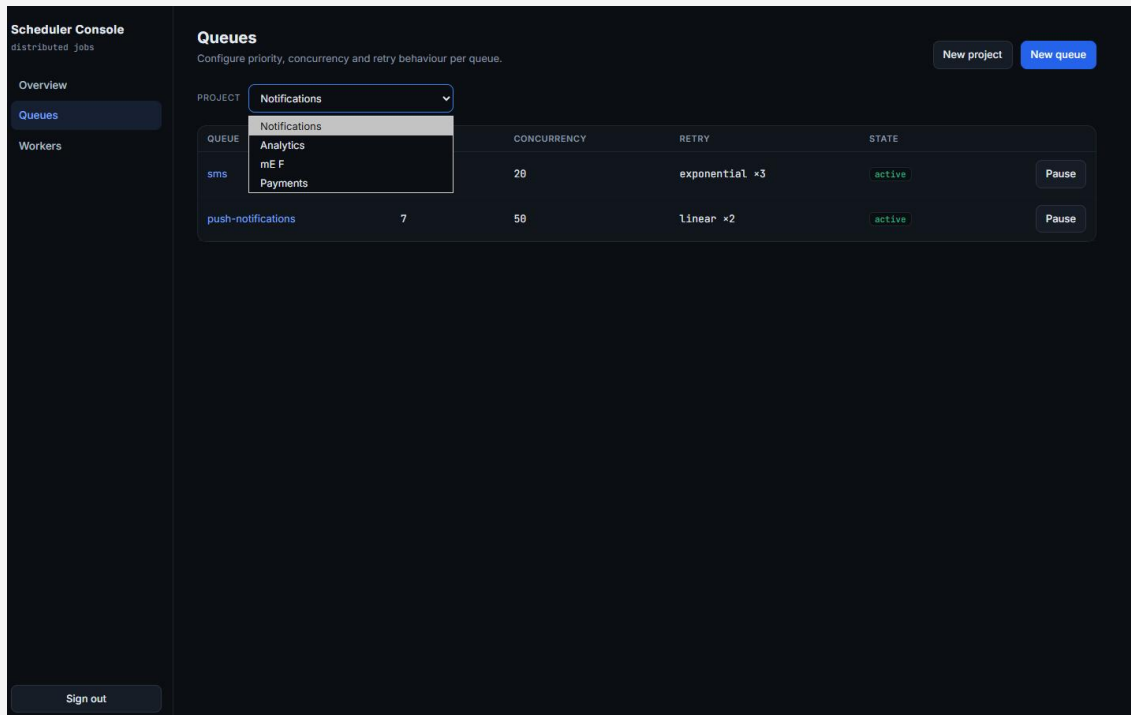


Figure 2: Queue Management

3. Enqueue Job

Job creation form showing all five job types: immediate, delayed, scheduled, recurring and batch

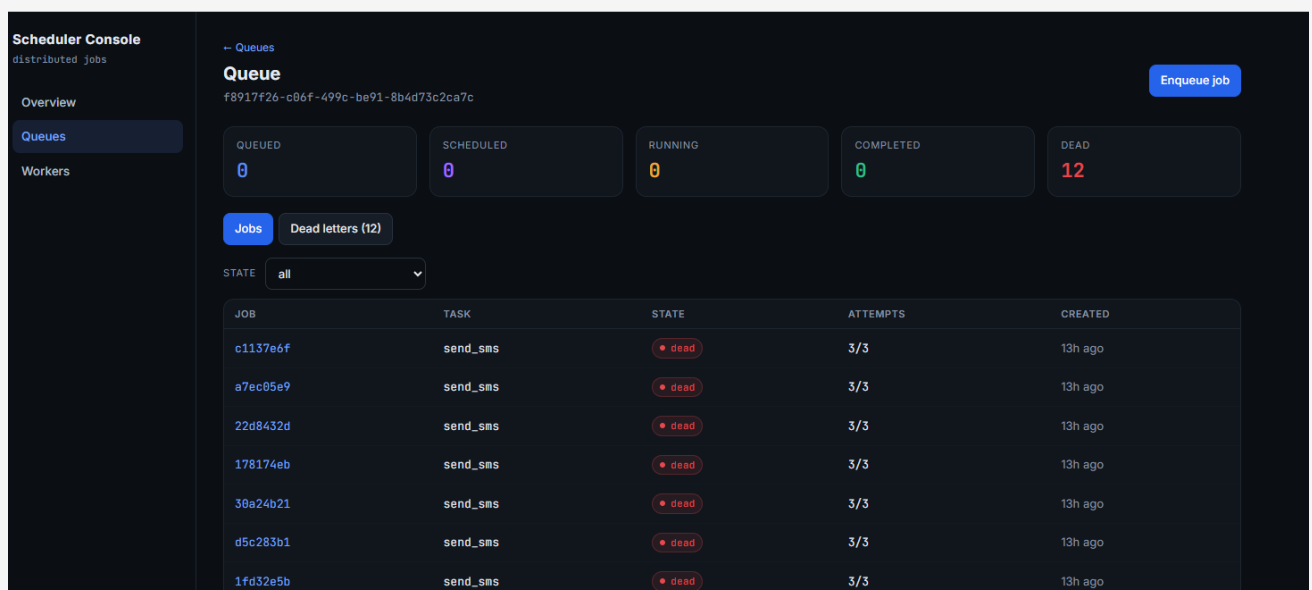


Figure 3: Enqueue Job

4. Job Detail View

Full job detail with execution history, per-attempt logs and result payload

The screenshot shows the 'Scheduler Console' interface with a sidebar containing 'Overview', 'Queues', and 'Workers'. The main content area displays details for a job named 'send_sms' with ID 'c1137e6f-3de9-4f6f-b174-cdcb6247dale'. The job status is 'dead'. A 'Retry' button is visible in the top right.

Details

Type	immediate
Priority	8
Attempts	3 / 3
Retry	exponential
Cron	
Worker	26584871
Created	13h ago
Finished	13h ago

Payload / result

PAYLOAD

```
{
  "phone": "+15550000011",
  "message": "Your code: 1011"
}
```

LAST ERROR

```
KeyError: "No handler registered for task 'send_sms'"
```

Execution history

ATTEMPT	STATUS	WORKER	DURATION	ERROR
#1	failed	26584871	30ms	KeyError: "No handler registered for task 'send_sms'"
#2	failed	26584871	126ms	KeyError: "No handler registered for task 'send_sms'"
#3	failed	26584871	334ms	KeyError: "No handler registered for task 'send_sms'"

Logs

```
13h ago info Attempt 1 started on worker 26584871-e208-4d89-b134-674a8cd955c5
13h ago warning Attempt 1 failed, retrying in 1.9s
13h ago info Attempt 2 started on worker 26584871-e208-4d89-b134-674a8cd955c5
13h ago warning Attempt 2 failed, retrying in 4.4s
13h ago info Attempt 3 started on worker 26584871-e208-4d89-b134-674a8cd955c5
13h ago error Job dead-lettered after 3 attempts
```

Sign out

Figure 4: Job Detail View

5. Worker Monitoring

Live worker fleet with heartbeat status, active job count and lifetime counters

The screenshot shows the 'Scheduler Console' interface with a sidebar containing 'Overview', 'Queues', and 'Workers'. The main content area displays the 'Workers' section with a subtitle: 'Live worker fleet. A worker is considered offline after 30s without a heartbeat.'

WORKER	HOST	STATUS	ACTIVE	PROCESSED	FAILED	LAST HEARTBEAT
89d6bd22	zenoXmachine	online	0/4	8	21	1s ago
404962fc	zenoXmachine	offline	0/4	1	3	12h ago
26584871	zenoXmachine	offline	0/4	26	139	12h ago

Figure 5: Worker Monitoring

6. Dead Letter Queue

Dead-lettered jobs with replay button for operator triage

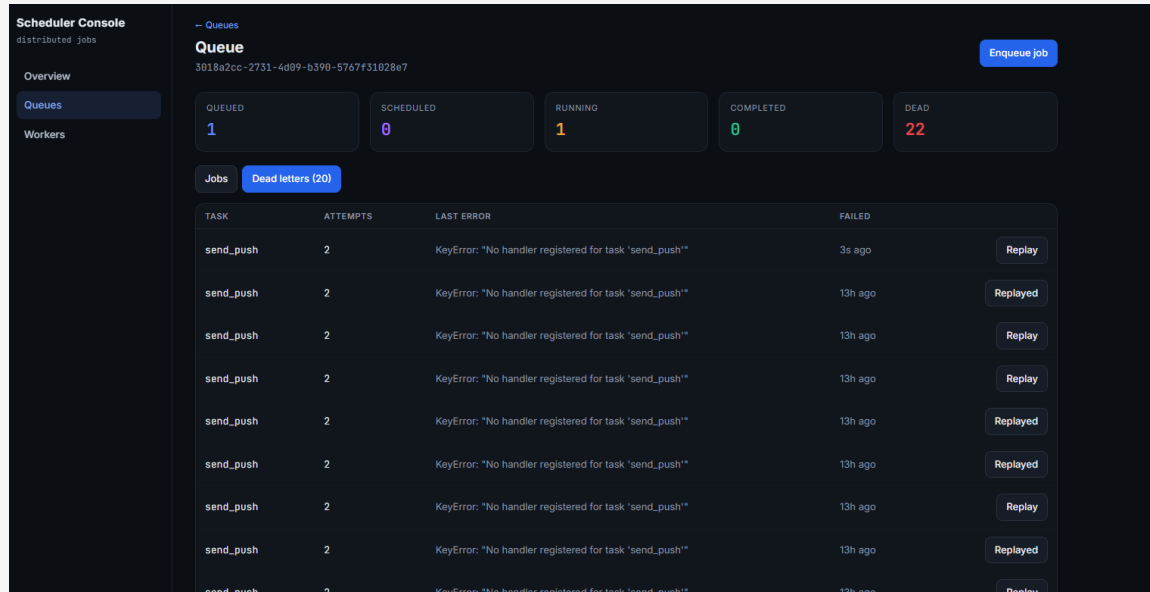


Figure 6: Dead Letter Queue

7. API Documentation

Interactive OpenAPI / Swagger UI at <http://localhost:8000/docs>

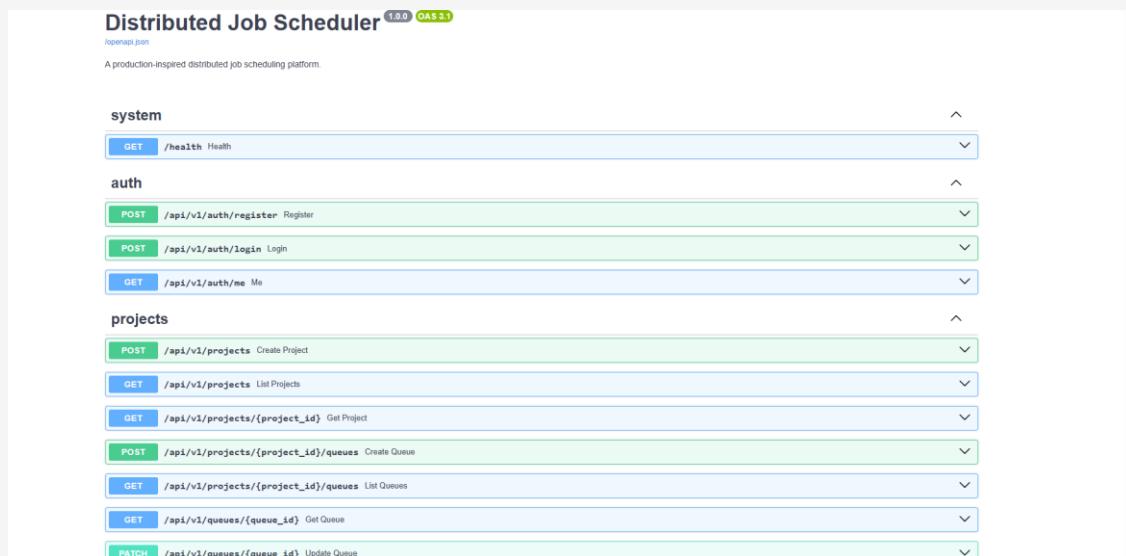


Figure 7: API Documentation

To capture screenshots: run the project using `run-local.bat`, open <http://localhost:5173> in your browser, seed demo data with the seed command, then capture each screen above and replace the placeholder boxes.